

Code Explanation

Utility Functions for Customer Order Processing

The provided code is a Python module (`src/utls.py`) containing utility functions for processing customer orders using Apache Spark. It includes functions to get a Spark session, validate data, and apply Slowly Changing Dimension (SCD) Type 2 changes to a target DataFrame.

Overview

The code is designed to facilitate data processing and management in a big data environment. It offers reusable functions that can be applied to various data processing tasks. The main functionality revolves around data validation and applying SCD Type 2 changes, which is a common requirement in data warehousing.

Step 1: Getting a Spark Session

This step involves obtaining a Spark session, which is the entry point to any Spark functionality. The `get\_spark\_session` function returns the active Spark session or creates a new one if it doesn't exist.

```
def get_spark_session() -> SparkSession:  
    return SparkSession.builder.getOrCreate()
```

Step 2: Validating Data

The `validate\_data` function checks if a given DataFrame contains all the required columns. It raises a `ValueError` if any required column is missing.

```
def validate_data(df, required_columns: List[str]) -> bool:  
    df_columns = df.columns  
    missing_columns = [col for col in required_columns if col not in  
df_columns]  
  
    if missing_columns:  
        raise ValueError(f"Missing required columns: {'', '}.join(miss  
ing_columns)}")  
  
    return True
```

Step 3: Applying SCD Type 2 Changes

The `apply\_scd\_type2\_changes` function is used to apply SCD Type 2 changes to a target DataFrame based on the data from a source DataFrame. It identifies records to be expired and inserted.

```
def apply_scd_type2_changes(  
    target_df,  
    source_df,  
    join_columns: List[str],  
    compare_columns: List[str]  
) -> Dict[str, Any]:  
    spark = get_spark_session()  
  
    # Only consider active records in the target  
    active_records = target_df.filter(col("IsActive") == True)
```

```

# Join source and target to find changes
joined = active_records.join(
    source_df,
    join_columns,
    "full_outer"
)

```

Step 4: Detecting Changes

This step involves creating a condition to detect changes between the source and target DataFrames based on the specified compare columns.

```

change_condition = None
for column in compare_columns:
    column_condition = (
        (active_records[column].isNull() & source_df[column].isNotNull()) |
        (active_records[column].isNotNull() & source_df[column].isNull()) |
        (active_records[column] != source_df[column])
    )

    if change_condition is None:
        change_condition = column_condition
    else:
        change_condition = change_condition | column_condition

```

Step 5: Identifying Records to Expire and Insert

The function identifies records that need to be expired (changed records in the target) and those that need to be inserted (new or changed records from the source).

```

records_to_expire = (
    joined
    .filter(change_condition)
    .filter(active_records[join_columns[0]].isNotNull())
    .select(active_records["*"])
    .withColumn("IsActive", lit(False))
    .withColumn("EndDate", current_timestamp())
)

records_to_insert = (
    joined
    .filter(
        change_condition |
        active_records[join_columns[0]].isNull()
    )
    .filter(source_df[join_columns[0]].isNotNull())
    .select(source_df["*"])
    .withColumn("IsActive", lit(True))
    .withColumn("StartDate", current_timestamp())
    .withColumn("EndDate", lit(None).cast("timestamp"))
)

```

Step 6: Returning the Results

The function returns a dictionary containing DataFrames for records to be expired and inserted.

```
return {  
    "to_expire": records_to_expire,  
    "to_insert": records_to_insert  
}
```